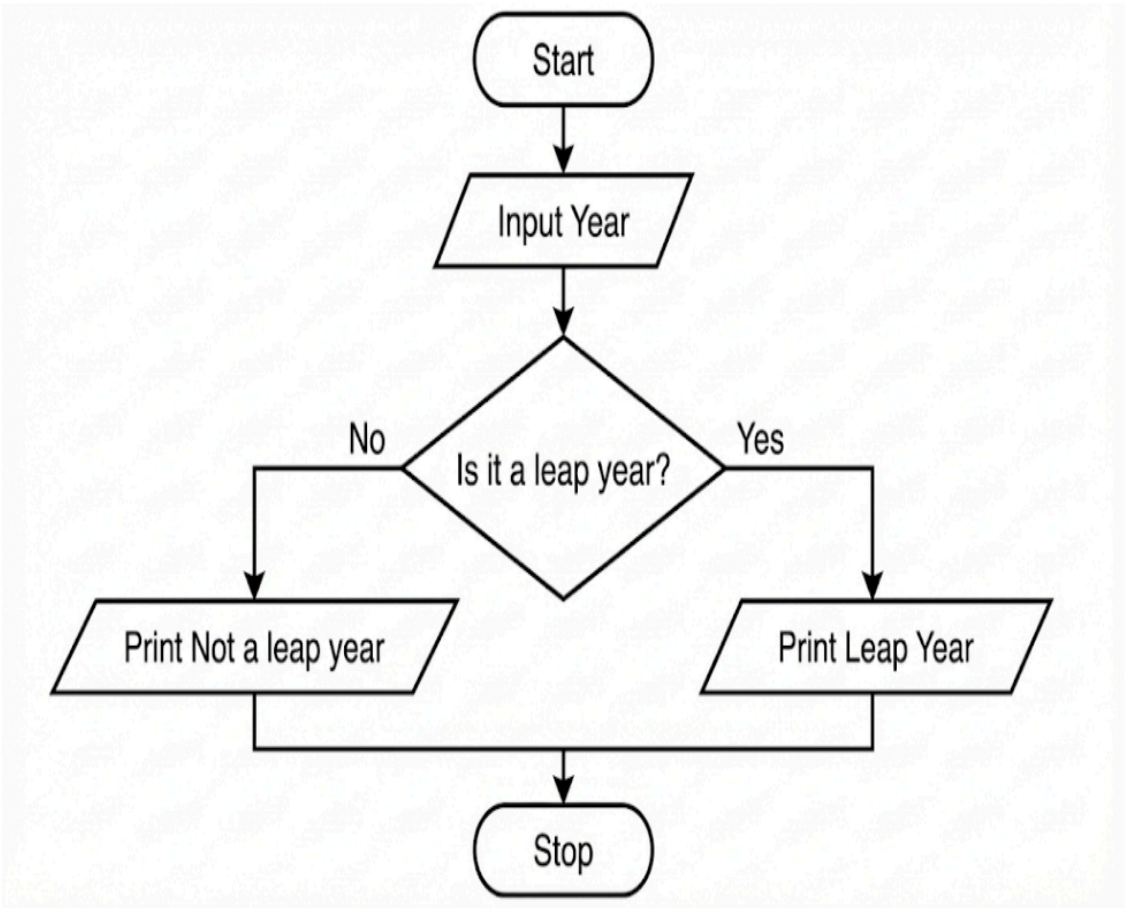
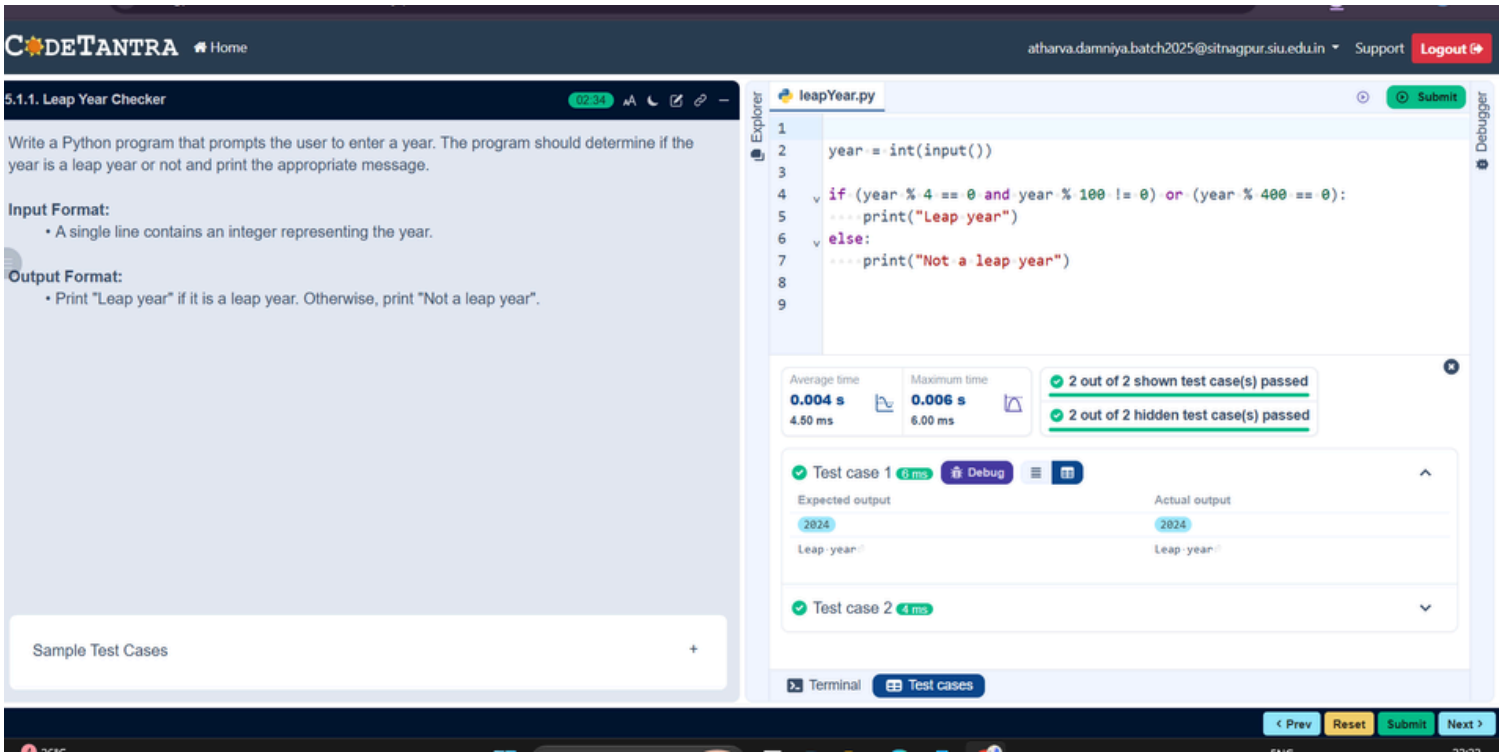
Experiment 5.1: Leap year checker
Algorithm:

```
Step 1: START
Step 2: INPUT year (integer)
Step 3: CHECK  IF (year % 4 == 0 AND year % 100 != 0)
               OR (year % 400 == 0)
           IF TRUE  → OUTPUT "Leap year"
           IF FALSE → OUTPUT "Not a leap year"
```

Flowchart:



Code:

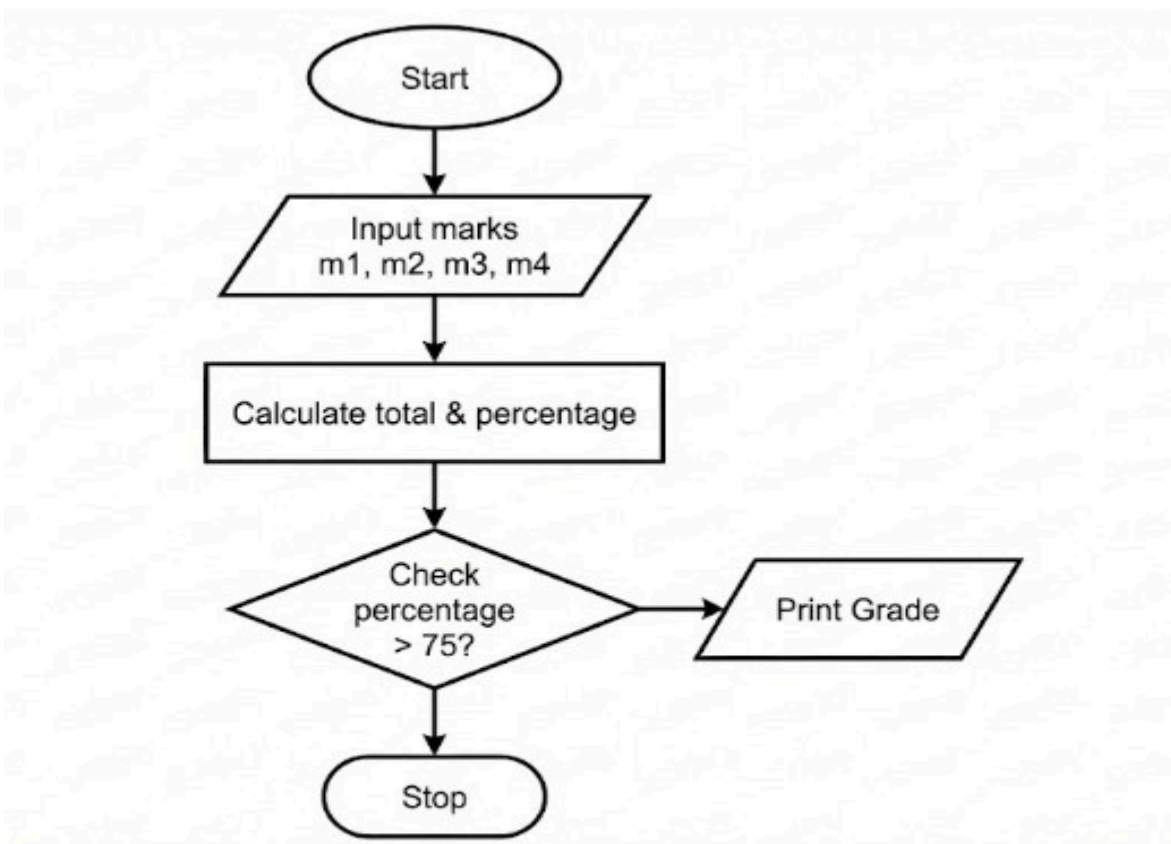

CODETANTRA
Home
atharva.damniya.batch2025@sitnagpur.siu.edu.in
Support
Logout
5.1.1. Leap Year Checker
02:34
Write a Python program that prompts the user to enter a year. The program should determine if the year is a leap year or not and print the appropriate message.
Input Format:
• A single line contains an integer representing the year.
Output Format:
• Print "Leap year" if it is a leap year. Otherwise, print "Not a leap year".
Sample Test Cases
Explorer
leapYear.py
Submit
Debugger
year = int(input())
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
print("Leap year")
else:
print("Not a leap year")
Average time
0.004 s
4.50 ms
Maximum time
0.006 s
6.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
6 ms
Debug
Expected output
2024
Leap year
Actual output
2024
Leap year
Test case 2
4 ms
Terminal
Test cases
Prev
Reset
Submit
Next

Experiment 5.2: Student grade based on aggregate

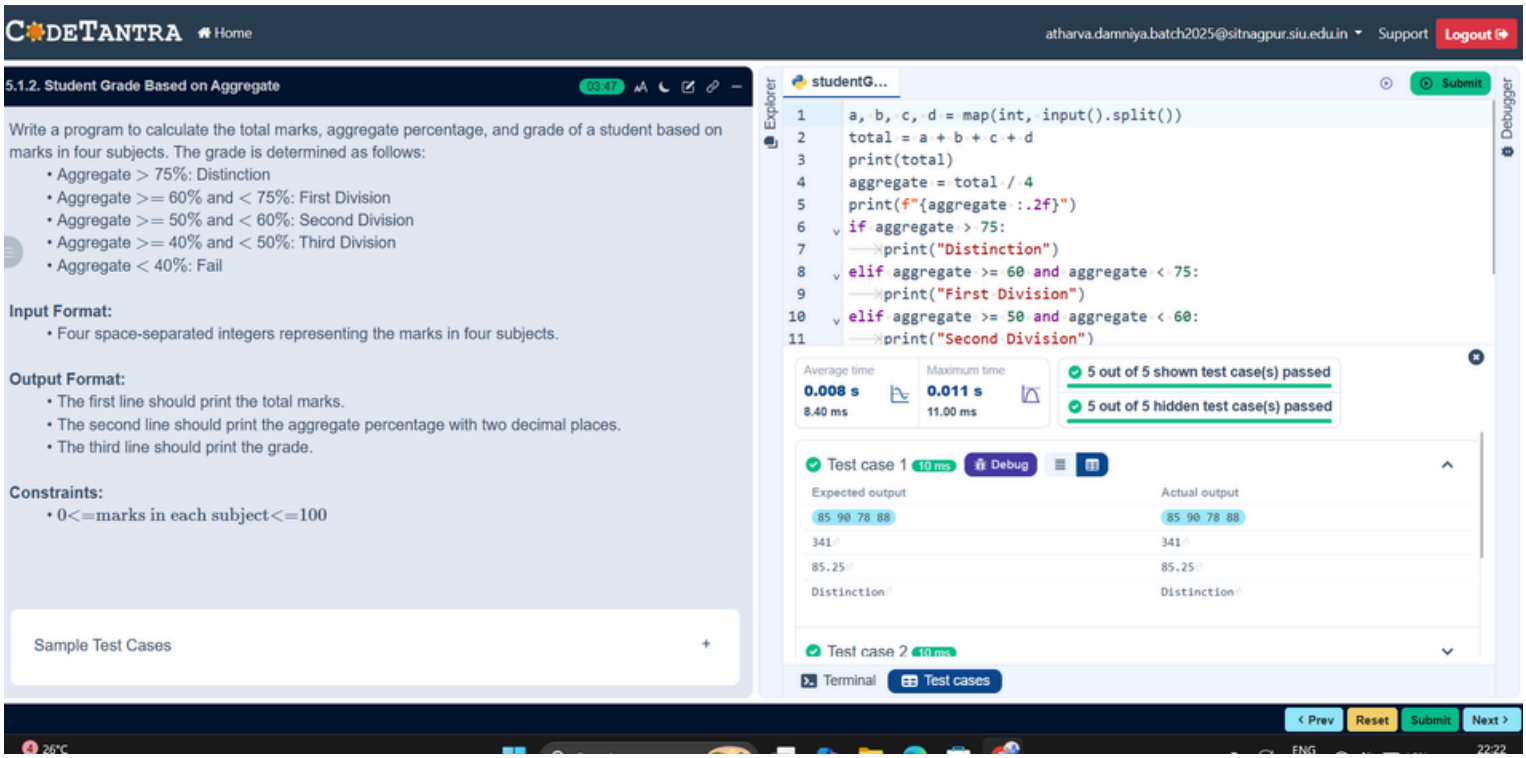
Algorithm:

```
Step 1: START
Step 2: INPUT m1, m2, m3, m4  (four integers, space-separated)
Step 3: CALCULATE  total     = m1 + m2 + m3 + m4
Step 4: CALCULATE  aggregate = total / 4
Step 5: OUTPUT total
Step 6: OUTPUT aggregate  (formatted to 2 decimal places)
Step 7: CHECK aggregate:
           IF aggregate >= 75            → OUTPUT "Distinction"
           ELSE IF aggregate >= 60       → OUTPUT "First Division"
           ELSE IF aggregate >= 50       → OUTPUT "Second Division"
           ELSE IF aggregate >= 40       → OUTPUT "Third Division"
           ELSE                          → OUTPUT "Fail"
Step 8: STOP
```

Flowchart:



Code:


CODETANTRA
Home
atharva.damniya.batch2025@sitnagpur.siu.edu.in
Support
Logout
5.1.2. Student Grade Based on Aggregate
03:47
Write a program to calculate the total marks, aggregate percentage, and grade of a student based on marks in four subjects. The grade is determined as follows:
• Aggregate > 75%: Distinction
• Aggregate >= 60% and < 75%: First Division
• Aggregate >= 50% and < 60%: Second Division
• Aggregate >= 40% and < 50%: Third Division
• Aggregate < 40%: Fail
Input Format:
• Four space-separated integers representing the marks in four subjects.
Output Format:
• The first line should print the total marks.
• The second line should print the aggregate percentage with two decimal places.
• The third line should print the grade.
Constraints:
• 0<=marks in each subject<=100
Sample Test Cases
Explorer
studentG...
Submit
Debugger
a, b, c, d = map(int, input().split())
total = a + b + c + d
print(total)
aggregate = total / 4
print(f"{aggregate :.2f}")
if aggregate > 75:
print("Distinction")
elif aggregate >= 60 and aggregate < 75:
print("First Division")
elif aggregate >= 50 and aggregate < 60:
print("Second Division")
Average time
0.008 s
8.40 ms
Maximum time
0.011 s
11.00 ms
5 out of 5 shown test case(s) passed
5 out of 5 hidden test case(s) passed
Test case 1
10 ms
Debug
Expected output
Actual output
85 90 78 88
85 90 78 88
341
341
85.25
85.25
Distinction
Distinction
Test case 2
10 ms
Terminal
Test cases
Prev
Reset
Submit
Next

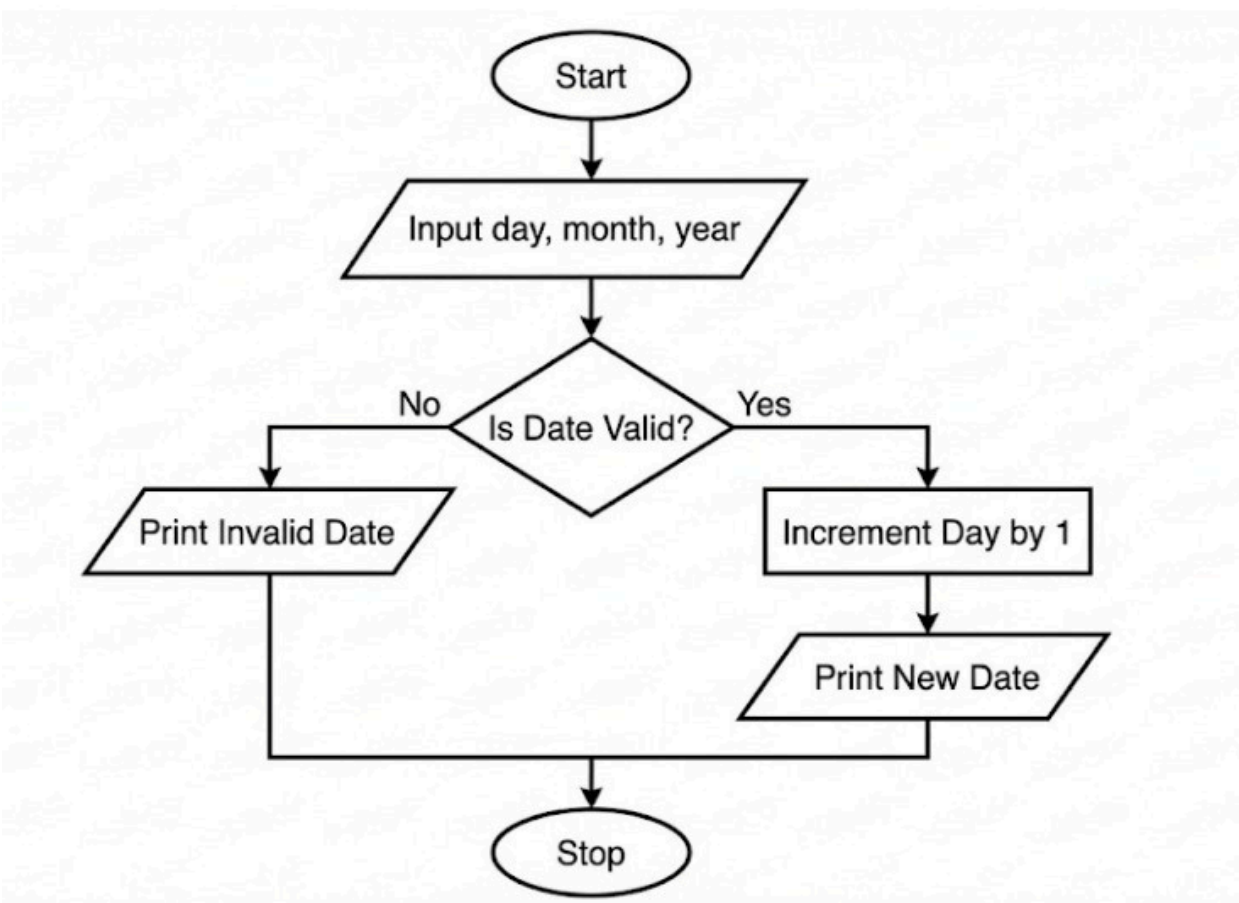
Experiment 6.1: Incremented Date

Algorithm:

```
Step 1: START
Step 2: INPUT day, month, year  (three separate integers)
Step 3: IF year <= 0 OR month < 1 OR month > 12
            → OUTPUT "Invalid Date" → STOP
Step 4: DETERMINE max_day:
           month in {1,3,5,7,8,10,12} → max_day = 31
           month in {4,6,9,11}        → max_day = 30
           month == 2:
               IF is_leap(year)        → max_day = 29
               ELSE                    → max_day = 28
Step 5: IF day < 1 OR day > max_day
            → OUTPUT "Invalid Date" → STOP
Step 6: INCREMENT:  day = day + 1
```

```
           IF day > max_day:  day = 1 ;  month = month + 1
               IF month > 12:  month = 1 ;  year = year + 1
Step 7: OUTPUT date as "DD-MM-YYYY"
Step 8: STOP
```

Flowchart:



Code:

CODETANTRA Home

atharva.damniya.batch2025@sitnagpur.siu.edu.in Support Logout

6.1.1. Incremented Date 01:11

Write a Python program to check if a given date is valid. If the date is valid, print the next day's date (incremented date). If the date is invalid, print "Invalid Date".

Date Validation Rules:

1. **Valid Month**: 1 to 12
2. **Valid Day**: Depends on the month and year
   - January (1), March (3), May (5), July (7), August (8), October (10), December (12): 1 to 31 days
   - April (4), June (6), September (9), November (11): 1 to 30 days
   - February (2): 1 to 29 days in a leap year and 1 to 28 days in a non-leap year
3. **Valid Year**: Any positive integer greater than zero.

Date Increment Rules:

If the date is valid, increment it by one day:

- If it's the last day of the month, move to the 1st day of the next month
- If it's December 31st, move to January 1st of the next year

Input Format:

- First line contains an integer representing the day
- Second line contains an integer representing the month
- Third line contains an integer representing the year

Sample Test Cases +

Explorer

nextDate.py

Submit

```
day = int(input())
month = int(input())
year = int(input())

flag = True

if year < 1 or month < 1 or month > 12:
    flag = False
else:
    if month in (1,3,5,7,8,10,12):
        maxdays = 31
```

Debugger

Average time 0.016 s 15.80 ms

Maximum time 0.061 s 61.00 ms

5 out of 5 shown test case(s) passed

5 out of 5 hidden test case(s) passed

Test case 1 12 ms Debug

Expected output	Actual output
15	15
3	3
2024	2024
16-03-2024	16-03-2024

Test case 2 12 ms

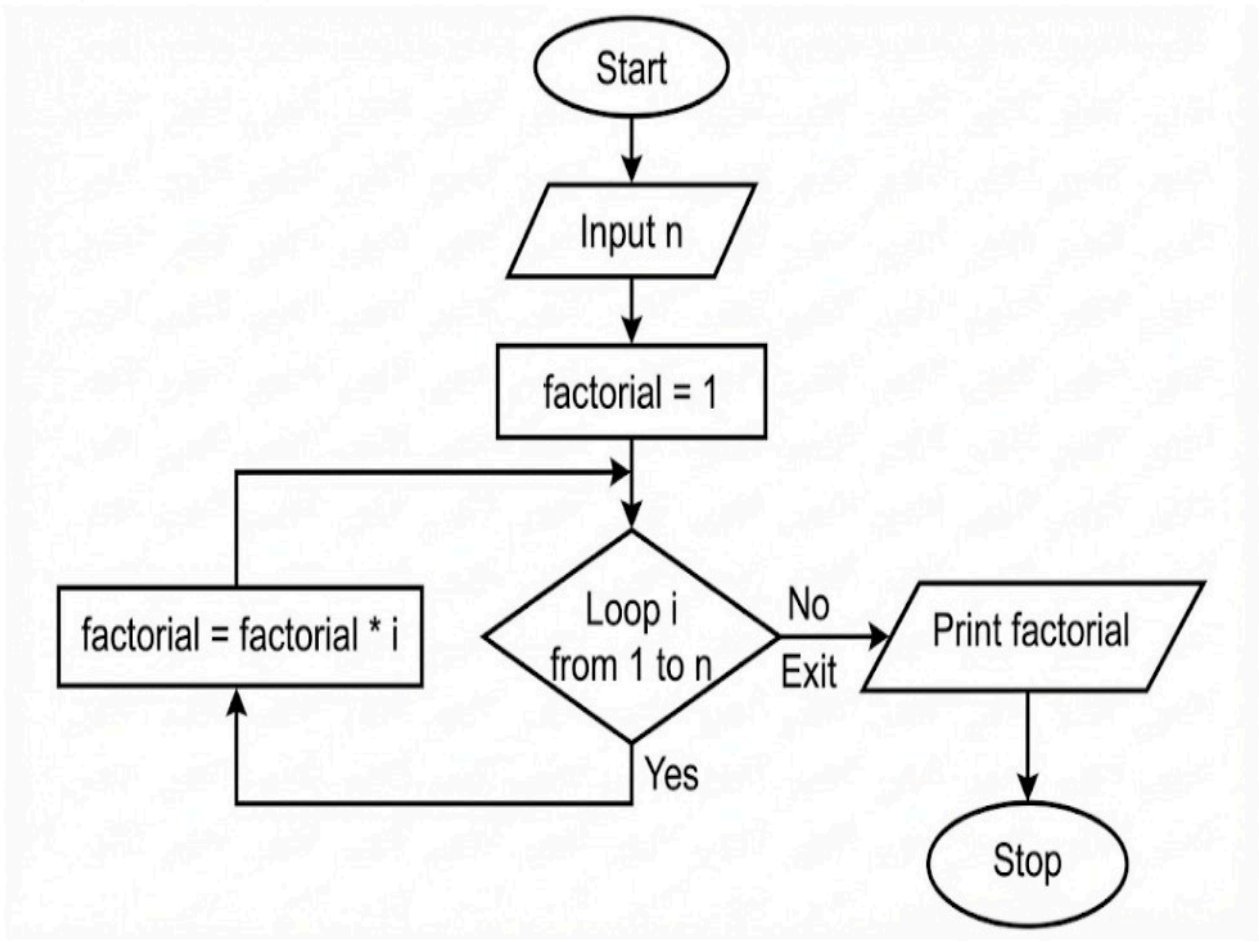
Terminal Test cases

< Prev Reset Submit Next >

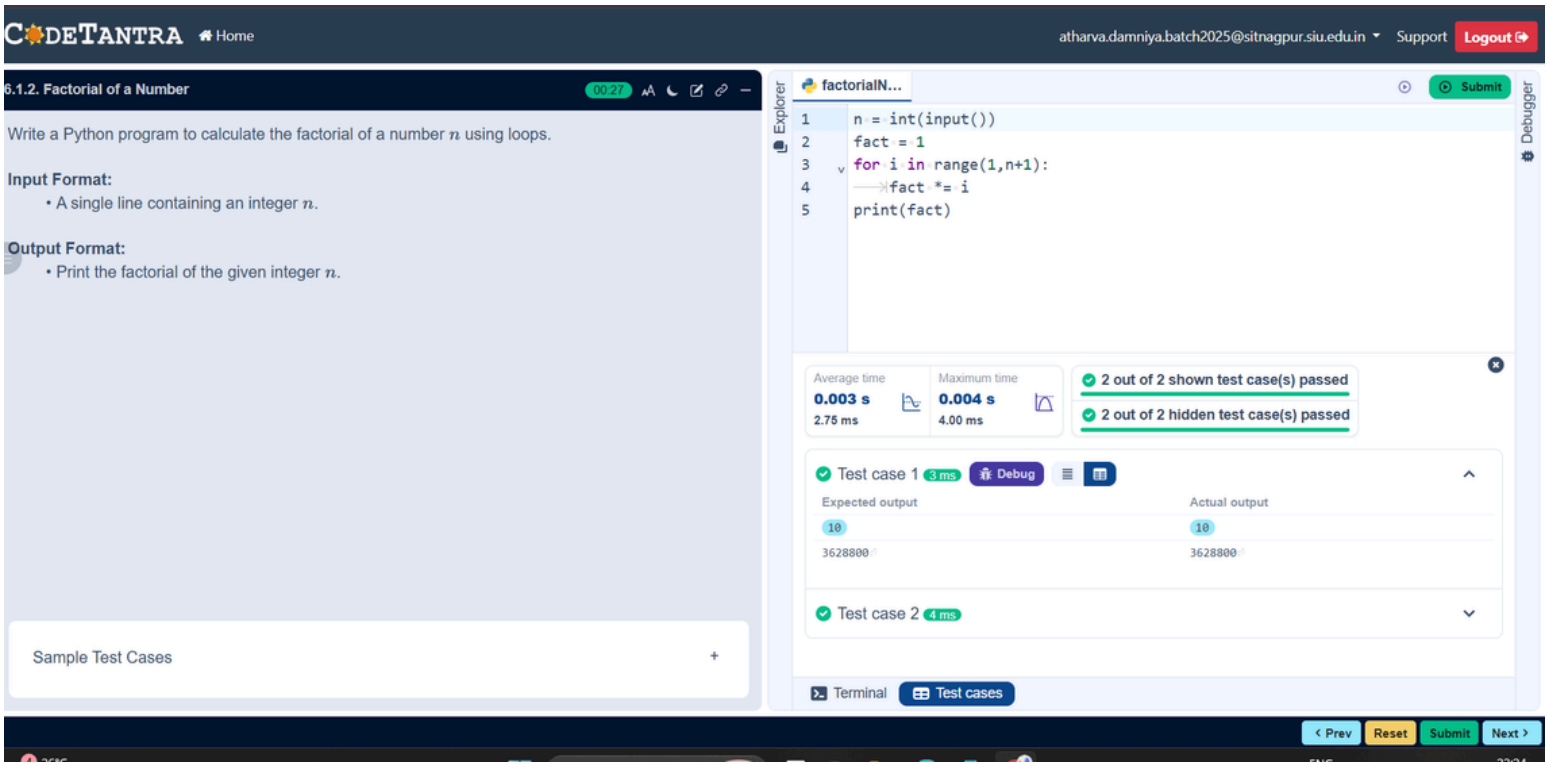
Experiment 6.2: Factorial of a number
Algorithm:

```
Step 1: START
Step 2: INPUT n  (integer)
Step 3: SET fact = 1
Step 4: FOR i = 1 TO n  (loop using range(1, n+1)):
            fact = fact × i
Step 5: OUTPUT fact
```

Flowchart:



Code:

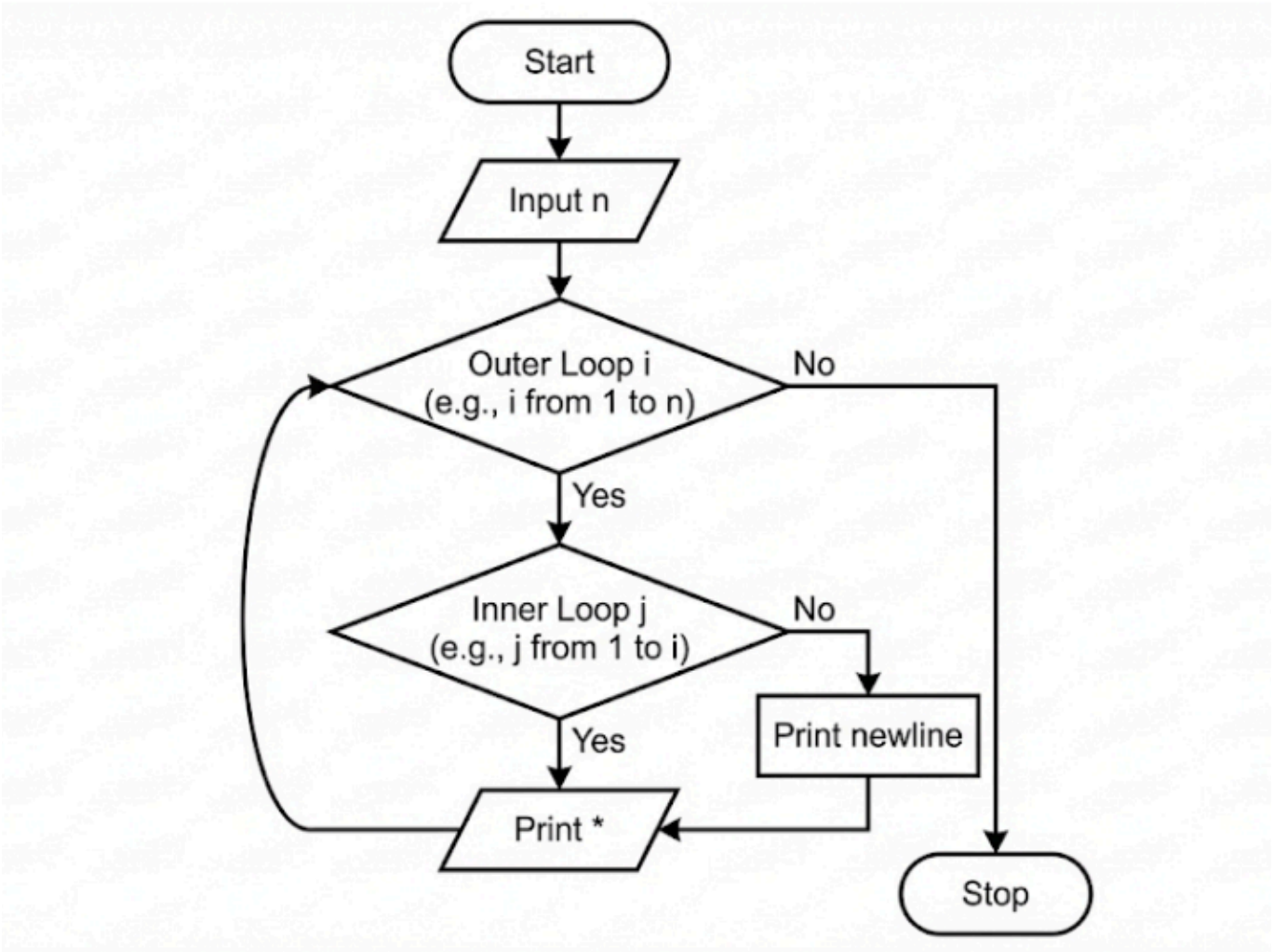

CODETANTRA
Home
atharva.damniya.batch2025@sitnagpur.siu.edu.in
Support
Logout
6.1.2. Factorial of a Number
00:27
Write a Python program to calculate the factorial of a number n using loops.
Input Format:
• A single line containing an integer n.
Output Format:
• Print the factorial of the given integer n.
Sample Test Cases
Explorer
factorialN...
Submit
Debugger
n = int(input())
fact = 1
for i in range(1,n+1):
fact *= i
print(fact)
Average time
0.003 s
2.75 ms
Maximum time
0.004 s
4.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
3 ms
Debug
Expected output
10
3628800
Actual output
10
3628800
Test case 2
4 ms
Terminal
Test cases
Prev
Reset
Submit
Next

Experiment 7.1: Inverted star pattern

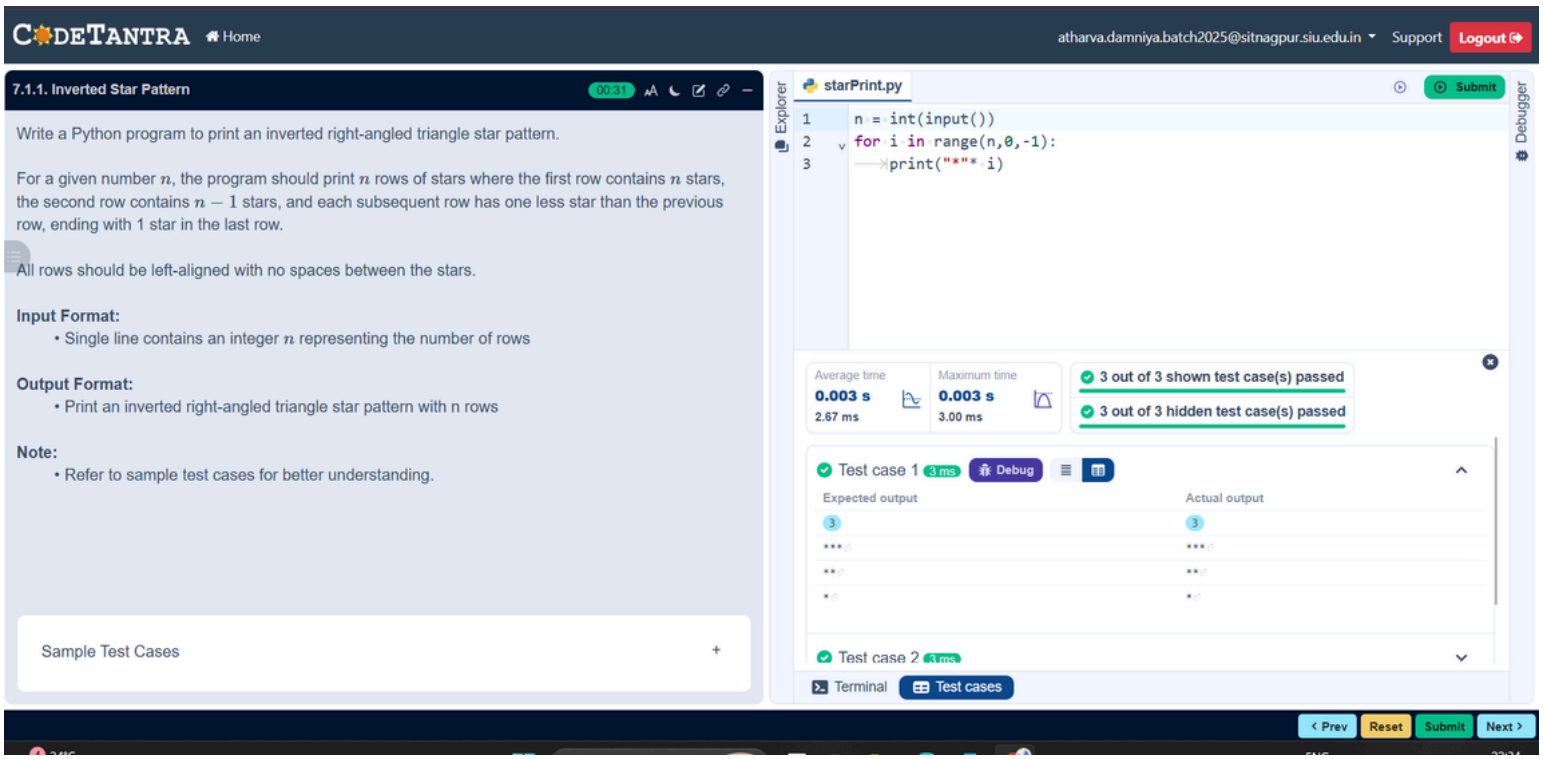
Algorithm:

```
Step 1: START
Step 2: INPUT n  (number of rows)
Step 3: FOR i = n DOWN TO 1  (range(n, 0, -1)):
            PRINT  '*' repeated i times  (no spaces)
Step 4: STOP
```

Flowchart:



Code:

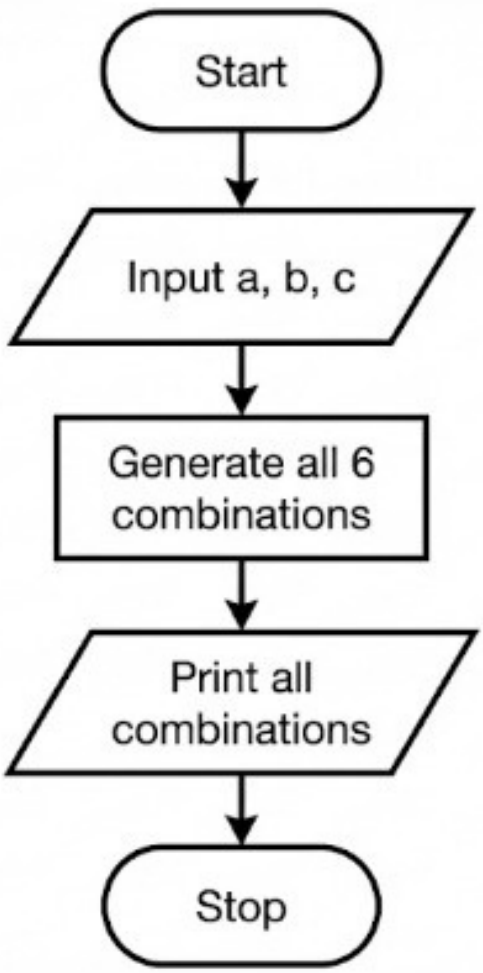

CODETANTRA
Home
atharva.damniya.batch2025@sitnagpur.siu.edu.in
Support
Logout
7.1.1. Inverted Star Pattern
00:31
Write a Python program to print an inverted right-angled triangle star pattern.
For a given number n, the program should print n rows of stars where the first row contains n stars, the second row contains $n-1$ stars, and each subsequent row has one less star than the previous row, ending with 1 star in the last row.
All rows should be left-aligned with no spaces between the stars.
Input Format:
• Single line contains an integer n representing the number of rows
Output Format:
• Print an inverted right-angled triangle star pattern with n rows
Note:
• Refer to sample test cases for better understanding.
Sample Test Cases
+
Explorer
starPrint.py
Submit
Debugger
1 n = int(input())
2 for i in range(n,0,-1):
3 print("*"* i)
Average time
0.003 s
2.67 ms
Maximum time
0.003 s
3.00 ms
3 out of 3 shown test case(s) passed
3 out of 3 hidden test case(s) passed
Test case 1
3 ms
Debug
Expected output
Actual output
3
3
Test case 2
Terminal
Test cases
Prev
Reset
Submit
Next

Experiment 7.2: Possible combination from 3 digits

Algorithm:

```
Step 1: START
Step 2: INPUT a, b, c  (three space-separated strings)
Step 3: STORE  digits = [a, b, c]
Step 4: FOR i = 0 TO 2:
            FOR j = 0 TO 2:
                FOR k = 0 TO 2:
                    IF i != j AND j != k AND i != k:
                        PRINT  digits[i] + digits[j] + digits[k]
Step 5: STOP
```

Flowchart:



Code:

CODETANTRA Home

atharva.damniya.batch2025@sitnagpur.siu.edu.in Support Logout

7.1.2. Possible Combinations from Three Digits 00:25

Write a Python program to accept three digits and print all possible 3-digit combinations that can be formed using those three digits. Each digit should be used exactly once in each combination.

For three digits a, b, and c, there are 6 possible arrangements based on their indices: abc, acb, bac, bca, cab, cba.

Input Format:

- Single line contains three space-separated non-negative integers, a, b, and c, representing the three digits

Output Format:

- Print all possible 3-digit combinations
- Each combination should be printed as a 3-digit number on a separate line
- Print combinations as they appear (maintain leading zeros if first digit is 0)

Constraints:

- $0 \leq \text{each digit} \leq 9$
- Digits can be same or different

Sample Test Cases +

Explorer

combinat...

Submit

Debugger

```python
a,b,c=input().split()
digits = [a,b,c]
for i in range(3):
    for j in range(3):
        for k in range(3):
            if i!=j and i!=k and j!=k:
                print(digits[i]+digits[j]+digits[k])
```

Average time **0.002 s** 2.14 ms

Maximum time **0.003 s** 3.00 ms

3 out of 3 shown test case(s) passed

4 out of 4 hidden test case(s) passed

Test case 1 3 ms Debug

Expected output	Actual output
1 2 3	1 2 3
123	123
132	132
213	213
231	231
312	312

Terminal Test cases

Prev Reset Submit Next